

PENGENALAN PYTHON

Sejarah Singkat Python

1. Awal Mula

- Python dikembangkan oleh **Guido van Rossum** pada akhir tahun **1980-an**.
- Guido bekerja di **Centrum Wiskunde & Informatica (CWI)**, sebuah lembaga penelitian di Belanda.
- Proyek ini dimulai pada **Desember 1989**, saat liburan Natal, sebagai “hobi” Guido untuk membuat bahasa pemrograman baru.

2. Tujuan Pembuatan Python

- Membuat bahasa pemrograman yang **mudah dibaca** dan **mudah digunakan**.
- Menjadi **alternatif dari ABC Language** (bahasa pemrograman sebelumnya di CWI) yang terlalu kaku.
- Menyediakan **fleksibilitas** seperti bahasa pemrograman populer (C, Modula-3) tetapi tetap sederhana.

3. Perilisan Resmi

Python 1.0 dirilis pada **1991**.

Fitur utamanya:

- Tipe data dasar (list, dict, str)
- Fungsi
- Error handling (exception)
- Modul (pemrograman terstruktur)

4. Perkembangan Versi

Python 2.0 (2000)

- List comprehension
- Garbage collection berbasis reference counting + cycle detector
- Unicode support (awal)

Python 3.0 (2008)

- Tidak backward compatible (berbeda dengan Python 2)

- print jadi fungsi (print())
- String default menjadi Unicode
- Lebih konsisten, mudah digunakan, dan modern

5. Python Sekarang

Python berkembang menjadi salah satu bahasa **paling populer di dunia**.

Dipakai untuk:

- **Data Science & AI** (Pandas, NumPy, TensorFlow, PyTorch)
- **Web Development** (Django, Flask, FastAPI)
- **Automasi & Scripting**
- **Cybersecurity & Hacking Tools**
- **Game Development** (Pygame)

6. Kenapa Namanya “Python”?

- Nama “Python” **bukan dari ular**, tetapi dari **grup komedi Inggris "Monty Python's Flying Circus"**.
- Guido van Rossum adalah penggemar acara tersebut dan ingin nama bahasa pemrogramannya **unik, singkat, dan mudah diingat**.

Kelebihan Python

Python menjadi salah satu bahasa pemrograman paling populer karena memiliki banyak keunggulan dibanding bahasa lain.

1. Mudah Dibaca & Dipelajari

- Sintaks Python sederhana, mirip dengan bahasa sehari-hari (bahasa Inggris).
 - Indentasi (spasi/tabs) digunakan untuk mengatur blok kode → lebih rapi dan jelas.
- Cocok untuk pemula yang baru belajar pemrograman.

```
# Contoh Python (mudah dipahami)
for i in range(1, 6):
    print("Angka ke-", i)
```

2. Memiliki Banyak Library & Framework

- Python memiliki **ribuan library** siap pakai, mulai dari standar hingga eksternal.

Beberapa contoh:

- **Data Science & AI** → NumPy, Pandas, TensorFlow, PyTorch

- **Web Development** → Django, Flask, FastAPI
- **Automasi & Scripting** → OS, Sys, Selenium
- **Visualisasi Data** → Matplotlib, Seaborn

b. Dengan library ini, kita bisa membangun aplikasi **lebih cepat tanpa menulis semua kode dari nol**.

3. Multi-Platform (Cross-Platform)

a. Python bisa dijalankan di berbagai sistem operasi:

- Windows
- Linux
- macOS
- Bahkan di Android (via Kivy, BeeWare) dan IoT (Raspberry Pi).

b. Program Python **portabel**, cukup tulis sekali, bisa dijalankan di banyak platform tanpa perubahan besar.

4. Komunitas Besar & Aktif

- Python memiliki komunitas global yang sangat besar.
- Dokumentasi lengkap, banyak forum (StackOverflow, Reddit, Github).
- Banyak kursus, buku, dan tutorial gratis.

5. Serbaguna

Python digunakan di berbagai bidang:

- Web development
- Data science
- Kecerdasan Buatan (AI & Machine Learning)
- Automasi
- Aplikasi desktop & mobile
- Game development

Ringkasan Kelebihan Python

- ✓ Mudah dipelajari → ramah pemula
- ✓ Library melimpah → mempercepat pengembangan

- ✓ Multi-platform → bisa jalan di banyak OS
- ✓ Komunitas besar → mudah cari Solusi
- ✓ Serbaguna → dipakai di berbagai bidang teknologi

Instalasi Python & IDE

1. Instalasi Python

- a. Kunjungi situs resmi: <https://www.python.org/downloads/>
- b. Pilih versi terbaru (saat ini biasanya Python 3.x).
- c. Jalankan installer:
 - Centang opsi **"Add Python to PATH"** → agar bisa dijalankan dari Command Prompt/Terminal.
 - Klik **Install Now**.

1. Cek instalasi:

```
python --version
```

Jika berhasil, akan muncul versi Python (contoh: Python 3.12.5).

2. IDE (Integrated Development Environment)

Python bisa dijalankan di berbagai IDE atau editor kode. Berikut beberapa pilihan populer:

a. IDLE (bawaan Python)

- Sudah otomatis terinstal bersama Python.
- Cocok untuk pemula.
- Bisa menulis kode langsung di Python Shell atau buat file baru (File → New File).

b. Visual Studio Code (VS Code)

- Editor modern, ringan, fleksibel.
- Install **Python Extension** dari marketplace.
- Mendukung debugging, auto-completion, dan integrasi Git.

c. PyCharm

- IDE khusus Python buatan JetBrains.
- Versi **Community** (gratis) atau **Professional** (berbayar).
- Banyak fitur untuk proyek besar (auto-import, refactoring, integrasi dengan framework).

d. Jupyter Notebook

- Sangat populer di **Data Science & Machine Learning**.
- Berbasis web (kode dijalankan dalam sel/blocks).
- Instalasi dengan:

```
pip install jupyter  
jupyter notebook
```



DASAR-DASAR PYTHON

Sintaks Dasar Python

Python punya aturan sintaks yang **sederhana** dibanding bahasa lain, tapi ada hal penting yang harus diperhatikan:

1. Indentasi (Indentation)

Python **tidak menggunakan tanda kurung {}** seperti C/Java untuk menandai blok kode, melainkan menggunakan **spasi atau tab**.

Aturan:

- Indentasi standar → **4 spasi** per level.
- Semua baris dalam satu blok harus sejajar.

Contoh benar:

```
if True:
    print("Baris ini masuk ke dalam blok IF")
    print("Indentasi harus konsisten")
```

Contoh salah (akan error):

```
if True:
print("Ini akan error!") # Tidak ada indentasi
```

Output error:

IndentationError: expected an indented block

2. Komentar (Comments)

Komentar digunakan untuk memberi catatan dalam kode, **tidak akan dijalankan oleh program**.

a. Komentar satu baris

Gunakan tanda pagar #

```
# Ini adalah komentar satu baris
print("Hello") # Bisa juga di akhir baris kode
```

b. Komentar multi-baris

Gunakan tiga tanda kutip `""" ... """` atau `""" ... """`

Biasanya dipakai untuk **dokumentasi**.

```
"""  
Program Python pertama  
Dibuat untuk belajar komentar  
"""  
  
print("Belajar Python")
```

3. Case Sensitive

Python membedakan huruf **besar dan kecil**.

```
nama = "Andi"  
print(nama) # benar  
print>Nama) # error: NameError
```

Tipe Data di Python

Python punya beberapa **tipe data dasar** yang sering dipakai:

1. Integer (int)

Bilangan bulat (positif, negatif, atau nol).

```
x = 10  
y = -5  
z = 0  
  
print(type(x)) # <class 'int'>
```

2. Floating Point (float)

Bilangan desimal / pecahan.

```
a = 3.14  
b = -2.5  
c = 0.0  
  
print(type(a)) # <class 'float'>
```

3. String (str)

Kumpulan karakter (huruf, angka, simbol) yang diapit tanda kutip ' ' atau " ".

```
nama = "Andi"
pesan = 'Belajar Python'
angka = "123" # meskipun terlihat angka, tetap string

print(type(nama)) # <class 'str'>
```

String bisa dioperasikan:

```
print("Halo " + "Dunia") # konkatenasi
print("Python " * 3)     # pengulangan
```

4. Boolean (bool)

Hanya punya dua nilai: **True** atau **False**.

Biasanya dipakai dalam **logika & perbandingan**.

```
benar = True
salah = False

print(type(benar)) # <class 'bool'>

# Contoh logika
print(10 > 5) # True
print(3 == 7) # False
```

5. Konversi Tipe Data

Python bisa mengubah satu tipe ke tipe lain (type casting).

```
# int -> float
x = float(5)    # 5.0

# float -> int
y = int(3.9)    # 3 (dibulatkan ke bawah)

# int/float -> str
z = str(100)    # "100"
```



```
# str -> int  
w = int("50")    # 50
```

Variabel dan Aturan Penamaan

1. Apa itu Variabel?

Variabel adalah **wadah untuk menyimpan data** di memori komputer.

Dalam Python, variabel **tidak perlu dideklarasikan tipe datanya** → otomatis sesuai dengan nilai yang disimpan (**dynamic typing**).

Contoh:

```
nama = "Andi"    # string  
umur = 17        # integer  
tinggi = 170.5   # float  
siswa = True     # boolean  
  
print(nama, umur, tinggi, siswa)
```

2. Aturan Penamaan Variabel

Python punya aturan khusus untuk memberi nama variabel:

Boleh:

- Huruf (a-z, A-Z), angka (0-9), dan garis bawah (_).
- Tidak boleh diawali angka.
- Nama variabel **case-sensitive** (huruf besar ≠ huruf kecil).

Contoh valid:

```
nama = "Budi"  
umur17 = 17  
tinggi_badan = 160
```

Tidak boleh:

```
2nama = "Salah"    # diawali angka  
nama siswa = "Salah" # ada spasi  
for = "Salah"      # pakai keyword Python
```

3. Konvensi Penulisan (Best Practice)

Untuk mempermudah dibaca, biasanya dipakai gaya penulisan berikut:

- **snake_case** → dipakai di Python (contoh: nama_lengkap)
- **UPPER_CASE** → untuk konstanta (contoh: PI = 3.14)
- **camelCase** → jarang dipakai di Python, lebih sering di Java/JavaScript.

4. Multiple Assignment

Python bisa memberikan nilai ke beberapa variabel sekaligus.

```
# Satu nilai untuk banyak variabel
x = y = z = 10
print(x, y, z) # 10 10 10

# Beberapa nilai ke beberapa variabel
a, b, c = 1, 2, 3
print(a, b, c) # 1 2 3
```

5. Menghapus Variabel

Variabel bisa dihapus dengan del.

```
x = 100
print(x) # 100

del x
# print(x) # Error: NameError
```

Operator di Python

Operator digunakan untuk melakukan operasi pada **variabel dan nilai**.

1. Operator Aritmatika

Digunakan untuk operasi matematika dasar.

Operator	Fungsi	Contoh	Hasil
+	Penjumlahan	5 + 3	8
-	Pengurangan	10 - 4	6
*	Perkalian	6 * 7	42

/	Pembagian (hasil float)	10 / 3	3.333...
//	Pembagian bulat	10 // 3	3
%	Sisa bagi (modulus)	10 % 3	1
**	Pangkat (eksponensial)	2 ** 3	8

Contoh:

```
a, b = 10, 3
print(a + b) # 13
print(a - b) # 7
print(a * b) # 30
print(a / b) # 3.333...
print(a // b) # 3
print(a % b) # 1
print(a ** b) # 1000
```

2. Operator Perbandingan (Relational Operators)

Digunakan untuk membandingkan dua nilai. Hasilnya **boolean** (True / False).

Operator	Fungsi	Contoh	Hasil
==	Sama dengan	5 == 5	True
!=	Tidak sama dengan	5 != 3	True
>	Lebih besar	7 > 4	True
<	Lebih kecil	2 < 8	True
>=	Lebih besar sama dengan	5 >= 5	True
<=	Lebih kecil sama dengan	3 <= 4	True

Contoh kode:

```
x, y = 5, 8
print(x == y) # False
print(x != y) # True
print(x > y) # False
print(x < y) # True
print(x >= 5) # True
print(y <= 10) # True
```

3. Operator Logika

Digunakan untuk menggabungkan ekspresi logika.

Operator	Fungsi	Contoh	Hasil
and	True jika kedua ekspresi True	(5 > 3) and (8 > 6)	True
or	True jika salah satu ekspresi True	(5 > 10) or (8 > 6)	True
not	Membalikkan nilai logika	not (5 > 3)	False

Contoh :

```
a, b = True, False
print(a and b) # False
print(a or b)  # True
print(not a)   # False
```

4. Operator Assignment

Digunakan untuk memberi / memperbarui nilai variabel.

Operator	Contoh	Sama dengan
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 2	x = x - 2
*=	x *= 4	x = x * 4
/=	x /= 5	x = x / 5
//=	x //= 2	x = x // 2
%=	x %= 2	x = x % 2
**=	x **= 3	x = x ** 3

Contoh :

```
x = 10
x += 5 # x = 15
x -= 3 # x = 12
x *= 2 # x = 24
x /= 4 # x = 6.0
print(x)
```

INPUT & OUTPUT

a. Fungsi input() → Mengambil Data dari Pengguna

- input() digunakan untuk menerima masukan dari user melalui keyboard.
- Data yang diterima **selalu berupa string (str)**, jadi jika butuh angka harus dikonversi.

Contoh:

```
nama = input("Masukkan nama kamu: ")
print("Halo,", nama)
```

Jika butuh angka:

```
umur = int(input("Masukkan umur: ")) # konversi ke int
tinggi = float(input("Masukkan tinggi (cm): ")) # konversi ke float
print("Umur:", umur)
print("Tinggi:", tinggi)
```

b. Fungsi print() → Menampilkan Data ke Layar

- Digunakan untuk mencetak teks atau variabel.
- Bisa mencetak lebih dari satu data sekaligus.

Contoh:

```
nama = "Budi"
umur = 18
print("Nama:", nama, "Umur:", umur)
```

Output:

```
Nama: Budi Umur: 18
```

c. F-String & Format String

Untuk menampilkan teks dengan variabel secara lebih rapi.

a. F-string (Python 3.6+)

- Gunakan f""" lalu {} untuk menaruh variabel.

```
nama = "Andi"
umur = 20
```

```
print(f'Halo, nama saya {nama} dan umur saya {umur} tahun.')
```

b. str.format() (cara lama, tapi masih dipakai)

```
nama = "Siti"  
umur = 22  
print("Halo, nama saya {} dan umur saya {} tahun.".format(nama, umur))
```

Output:

```
Halo, nama saya Siti dan umur saya 22 tahun.
```

c. Format dengan nomor urut / nama variabel

```
# Dengan index  
print("Umur {1} tahun, nama {0}".format("Ali", 25))  
  
# Dengan nama variabel  
print("Nama: {n}, Umur: {u}".format(n="Dina", u=30))
```

Ringkasan Input & Output

- `input()` → ambil data dari user (default string)
- `print()` → tampilkan data ke layar
- **F-string** (`f'...'`) → cara modern, cepat, mudah dibaca
- **format()** → cara lama, masih digunakan

STRUKTUR KONTROL

Percabangan (if, elif, else)

Percabangan digunakan untuk **mengambil keputusan** berdasarkan suatu kondisi.

1. Struktur Dasar if

```
if kondisi:
    # blok kode yang dijalankan jika kondisi True
```

Contoh:

```
umur = 18
if umur >= 17:
    print("Kamu sudah dewasa")
```

Output:

Kamu sudah dewasa

2. if ... else

Jika kondisi **True**, jalankan blok if. Jika **False**, jalankan blok else.

```
umur = 15
if umur >= 17:
    print("Kamu sudah dewasa")
else:
    print("Kamu masih remaja")
```

Output:

Kamu masih remaja

3. if ... elif ... else

Jika ada **lebih dari satu kondisi**, gunakan elif (else if).

```
nilai = 75

if nilai >= 90:
```

```
print("Grade A")
elif nilai >= 80:
    print("Grade B")
elif nilai >= 70:
    print("Grade C")
else:
    print("Grade D")
```

Output:

Grade C

4. Percabangan Bersarang (Nested if)

if bisa ditulis di dalam if lain.

```
umur = 20
status = "pelajar"

if umur >= 17:
    if status == "pelajar":
        print("Kamu pelajar dewasa")
    else:
        print("Kamu bukan pelajar, tapi sudah dewasa")
else:
    print("Kamu masih anak-anak")
```

Output:

Kamu pelajar dewasa

5. Percabangan Singkat (Ternary Operator)

Python mendukung penulisan if singkat:

```
umur = 18
status = "Dewasa" if umur >= 17 else "Anak-anak"
print(status)
```

Output:

Dewasa

Ringkasan Percabangan di Python

- if → kondisi tunggal
- if ... else → 2 kondisi (benar/salah)
- if ... elif ... else → lebih dari 2 kondisi
- Nested if → if di dalam if
- Ternary operator → if singkat

Perulangan (Looping)

Perulangan digunakan untuk menjalankan **blok kode secara berulang** selama kondisi tertentu terpenuhi.

1. Perulangan for

Digunakan untuk **mengulang sejumlah elemen** (iterasi).

a. For dengan range()

```
for i in range(5): # 0,1,2,3,4
    print("Perulangan ke-", i)
```

Output:

Perulangan ke- 0

Perulangan ke- 1

Perulangan ke- 2

Perulangan ke- 3

Perulangan ke- 4

range(start, stop, step)

- range(5) → 0 s/d 4
- range(1, 6) → 1 s/d 5
- range(2, 10, 2) → 2, 4, 6, 8

b. For dengan List / String

```
buah = ["apel", "mangga", "jeruk"]
for b in buah:
    print("Saya suka", b)
```

```
for huruf in "Python":  
    print(huruf)
```

2. Perulangan while

Digunakan jika **jumlah perulangan belum pasti** → selama kondisi True.

```
x = 1  
while x <= 5:  
    print("Nilai x =", x)  
    x += 1
```

Output:

Nilai x = 1
Nilai x = 2
Nilai x = 3
Nilai x = 4
Nilai x = 5

3. Kontrol dalam Perulangan

a. **break** → menghentikan perulangan

```
for i in range(1, 10):  
    if i == 5:  
        break  
    print(i)
```

Output: 1 2 3 4

b. **continue** → skip iterasi berikutnya

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```

Output: 1 2 4 5

c. **else pada loop**

else akan dijalankan jika loop selesai **tanpa break**.

```
for i in range(3):  
    print("Perulangan ke-", i)
```

```
else:  
    print("Loop selesai")
```

Output:

Perulangan ke- 0

Perulangan ke- 1

Perulangan ke- 2

Loop selesai

Ringkasan Perulangan Python

- **for** → digunakan untuk iterasi (range, list, string, dsb)
- **while** → digunakan selama kondisi True
- **break** → hentikan loop
- **continue** → lewati satu iterasi
- **else** → dijalankan jika loop selesai normal

Kontrol Perulangan: break, continue, pass

Saat menggunakan **for** atau **while**, kadang kita perlu **mengatur alur perulangan**. Python menyediakan tiga keyword penting:

1. **break** → menghentikan perulangan

break digunakan untuk **keluar dari loop** meskipun kondisi loop belum selesai.

```
for i in range(1, 10):  
    if i == 5:  
        break # hentikan loop saat i = 5  
    print(i)
```

Output:

1
2
3
4

2. **continue** → lewati satu iterasi

continue digunakan untuk **melewati 1 iterasi** dan lanjut ke iterasi berikutnya.

```
for i in range(1, 6):  
    if i == 3:  
        continue # lewati angka 3  
    print(i)
```

Output:

1
2
4
5

3. **pass** → tidak melakukan apa-apa

pass adalah **placeholder** (pengganti blok kosong).

Biasanya digunakan saat kita ingin menulis struktur kode tapi belum diisi.

```
for i in range(1, 6):  
    if i == 3:  
        pass # tidak melakukan apa-apa  
    print(i)
```

Output:

1
2
3
4
5

Perbedaannya:

- **break** → hentikan loop total
- **continue** → lompat ke iterasi berikutnya
- **pass** → tidak melakukan apa-apa (hanya placeholder)

STRUKTUR DATA

List (Daftar)

Pengertian

List adalah **struktur data** di Python yang digunakan untuk menyimpan **banyak nilai** secara berurutan.

- Bisa berisi berbagai tipe data (int, str, float, bahkan list lain).
- Bersifat **mutable** (dapat diubah setelah dibuat).
- Ditulis dengan tanda `[ ]`.

Membuat List

```
buah = ["apel", "mangga", "jeruk"]  
angka = [1, 2, 3, 4, 5]  
campuran = [1, "Python", 3.14, True]
```

Mengakses Elemen List

Menggunakan **indeks** (dimulai dari 0).

```
buah = ["apel", "mangga", "jeruk"]  
  
print(buah[0]) # apel  
print(buah[1]) # mangga  
print(buah[-1]) # jeruk (indeks negatif: dari belakang)
```

Mengubah Elemen List

```
buah[1] = "pisang"  
print(buah) # ['apel', 'pisang', 'jeruk']
```

Menambahkan Elemen

```
buah.append("manggis") # tambah di akhir  
buah.insert(1, "durian") # tambah di indeks 1
```

```
print(buah)
```

Output:

```
['apel', 'durian', 'pisang', 'jeruk', 'manggis']
```

Menghapus Elemen

```
buah.remove("pisang") # hapus berdasarkan nilai
buah.pop(0)           # hapus berdasarkan indeks
del buah[-1]          # hapus elemen terakhir
print(buah)
```

Fungsi & Operasi List

```
angka = [10, 20, 30, 40]

print(len(angka)) # 4 (jumlah elemen)
print(min(angka)) # 10
print(max(angka)) # 40
print(sum(angka)) # 100
```

Perulangan pada List

```
buah = ["apel", "mangga", "jeruk"]

for item in buah:
    print("Saya suka", item)
```

List Bersarang (Nested List)

```
data = [
    [1, 2, 3],
    ["a", "b", "c"]
]

print(data[0][1]) # 2
print(data[1][2]) # c
```

Ringkasan

- List ditulis dengan []
- Bisa diubah (mutable)
- Dapat berisi berbagai tipe data
- Operasi dasar: akses, ubah, tambah (append, insert), hapus (remove, pop, del)

Tuple di Python

Pengertian

Tuple adalah struktur data di Python yang mirip dengan **List**, tetapi:

- Ditulis dengan tanda ()
- Bersifat **immutable** → artinya **tidak bisa diubah** (elemen tidak dapat ditambah, dihapus, atau diganti setelah dibuat).

Cocok digunakan untuk **data tetap** yang tidak boleh dimodifikasi.

Membuat Tuple

```
buah = ("apel", "mangga", "jeruk")
angka = (1, 2, 3, 4, 5)
campuran = (1, "Python", 3.14, True)
```

Jika hanya **satu elemen**, harus pakai koma , supaya dianggap tuple:

```
satu = (5,)
print(type(satu)) # <class 'tuple'>
```

Mengakses Elemen Tuple

Sama seperti list, pakai **indeks**.

```
buah = ("apel", "mangga", "jeruk")

print(buah[0]) # apel
print(buah[-1]) # jeruk
```

Iterasi pada Tuple

```
for item in buah:
    print("Saya suka", item)
```

Tuple Tidak Bisa Diubah

```
buah = ("apel", "mangga", "jeruk")  
# buah[1] = "pisang" # ❌ Error (tuple immutable)
```

Fungsi Bawaan untuk Tuple

```
angka = (10, 20, 30, 40)  
  
print(len(angka)) # 4  
print(min(angka)) # 10  
print(max(angka)) # 40  
print(sum(angka)) # 100
```

Tuple Bersarang

```
data = (  
    (1, 2, 3),  
    ("a", "b", "c")  
)  
  
print(data[0][1]) # 2  
print(data[1][2]) # c
```

Tuple Unpacking

Membongkar tuple ke variabel secara langsung:

```
koordinat = (10, 20, 30)  
x, y, z = koordinat  
print(x, y, z) # 10 20 30
```

Ringkasan

- Tuple mirip List, tapi **immutable** (tidak bisa diubah).
- Digunakan untuk data yang sifatnya tetap.
- Mendukung indexing, slicing, looping, dan nested tuple.

Set di Python

Pengertian

Set adalah struktur data di Python yang digunakan untuk menyimpan **kumpulan elemen unik** (tidak ada duplikasi) dan **tidak berurutan** (unordered).

- Ditulis dengan tanda { }
- Tidak bisa diakses dengan indeks (karena tidak berurutan)
- Cocok dipakai untuk operasi **himpunan** (union, intersection, difference, dll).

Membuat Set

```
buah = {"apel", "mangga", "jeruk", "apel"}
print(buah)
# {'apel', 'mangga', 'jeruk'} → elemen unik, 'apel' hanya muncul sekali

kosong = set() # membuat set kosong
```

Menambah & Menghapus Elemen

```
buah = {"apel", "mangga", "jeruk"}

buah.add("pisang")    # tambah elemen
buah.remove("mangga") # hapus elemen tertentu (error kalau tidak ada)
buah.discard("durian") # hapus elemen tanpa error kalau tidak ada
buah.pop()            # hapus elemen acak (karena unordered)

print(buah)
```

Operasi Himpunan

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A | B) # Union → {1, 2, 3, 4, 5, 6}
print(A & B) # Intersection → {3, 4}
print(A - B) # Difference → {1, 2}
print(A ^ B) # Symmetric Difference → {1, 2, 5, 6}
```

Mengecek Keanggotaan

```
buah = {"apel", "mangga", "jeruk"}

print("apel" in buah)  # True
print("pisang" not in buah) # True
```

Iterasi pada Set

```
for item in buah:
    print(item)  # urutannya bisa berbeda-beda
```

Set dari List / Tuple

```
angka = [1, 2, 2, 3, 4, 4, 5]
unik = set(angka)
print(unik)  # {1, 2, 3, 4, 5}
```

Ringkasan

- Set = himpunan **unik** dan **tidak berurutan**
- Tidak bisa diakses pakai indeks
- Bisa menambah (add), hapus (remove, discard, pop)
- Mendukung operasi himpunan (union, intersection, difference, dll)

Dictionary di Python

Pengertian

Dictionary adalah struktur data di Python yang menyimpan data dalam bentuk **pasangan key-value**:

- **Key** → harus unik, biasanya berupa string atau angka
- **Value** → bisa berupa tipe data apa saja (int, str, list, dict, dll)
- Ditulis dengan tanda kurung kurawal { } dengan format key: value

Membuat Dictionary

```
mahasiswa = {
    "nama": "Budi",
    "nim": "12345",
```

```
"jurusan": "Informatika",  
"ipk": 3.75  
}  
  
print(mahasiswa["nama"]) # Budi  
print(mahasiswa["ipk"]) # 3.75
```

Mengakses Data

```
print(mahasiswa.get("jurusan")) # Informatika  
print(mahasiswa.get("alamat", "Tidak ada"))  
# jika key tidak ditemukan → kembalikan nilai default
```

Menambah & Mengubah Data

```
mahasiswa["email"] = "budi@mail.com" # menambah key baru  
mahasiswa["ipk"] = 3.90 # mengubah value
```

Menghapus Data

```
mahasiswa.pop("nim") # hapus berdasarkan key  
del mahasiswa["jurusan"] # hapus key-value  
mahasiswa.clear() # hapus semua data
```

Iterasi pada Dictionary

```
mahasiswa = {  
    "nama": "Budi",  
    "nim": "12345",  
    "jurusan": "Informatika"  
}  
  
for key in mahasiswa:  
    print(key, ":", mahasiswa[key])  
  
# atau  
for key, value in mahasiswa.items():
```

```
print(key, "→", value)
```

Fungsi Bawaan Dictionary

```
print(mahasiswa.keys()) # semua key  
print(mahasiswa.values()) # semua value  
print(mahasiswa.items()) # semua pasangan key-value
```

Nested Dictionary (Dictionary Bersarang)

```
data_mahasiswa = {  
    "mhs1": {"nama": "Budi", "ipk": 3.9},  
    "mhs2": {"nama": "Siti", "ipk": 3.7}  
}  
  
print(data_mahasiswa["mhs1"]["nama"]) # Budi
```

Ringkasan

- Dictionary = struktur data dengan pasangan **key** → **value**
- Key harus unik, value bebas
- Mendukung operasi tambah, ubah, hapus, iterasi
- Bisa digunakan untuk data kompleks (nested dictionary)

FUNGSI

Pengertian

Fungsi adalah blok kode yang dapat dipanggil berkali-kali untuk menjalankan tugas tertentu.

Tujuan penggunaan fungsi:

- Menghindari pengulangan kode
- Membuat program lebih rapi & modular
- Mudah dikelola dan digunakan kembali (reusable)

Membuat Fungsi dengan def

Sintaks dasar:

```
def nama_fungsi(parameter1, parameter2, ...):  
    # blok kode  
    return nilai
```

Contoh Fungsi Sederhana

```
def sapa():  
    print("Halo, selamat datang di Python!")  
  
sapa() # memanggil fungsi
```

Output:

Halo, selamat datang di Python!

Fungsi dengan Parameter

```
def sapa_nama(nama):  
    print(f'Halo {nama}, selamat belajar Python!')  
  
sapa_nama("Budi")  
sapa_nama("Siti")
```

Output:

Halo Budi, selamat belajar Python!

Halo Siti, selamat belajar Python!

Fungsi dengan Nilai Kembali (return)

```
def tambah(a, b):  
    return a + b  
  
hasil = tambah(5, 3)  
print("Hasil penjumlahan:", hasil)
```

Output:

Hasil penjumlahan: 8

Parameter Default

```
def sapa(nama="User"):  
    print(f'Halo {nama}!')  
  
sapa()    # tanpa argumen  
sapa("Andi") # dengan argumen
```

Output:

Halo User!

Halo Andi!

Fungsi dengan Banyak Parameter (args & kwargs)

```
def jumlahkan(*angka): # *args → banyak parameter  
    return sum(angka)  
  
print(jumlahkan(1, 2, 3, 4, 5)) # 15  
  
def biodata(**data): # **kwargs → pasangan key-value  
    for key, value in data.items():  
        print(f'{key} : {value}')  
  
biodata(nama="Siti", umur=20, jurusan="TI")
```

Output:

15

nama : Siti

umur : 20

jurusan : TI

Ringkasan

- Fungsi didefinisikan dengan def
- Bisa menerima parameter & mengembalikan nilai dengan return
- Bisa pakai **default parameter**, *args, **kwargs

Parameter & Return Value pada Fungsi

Parameter

- **Parameter** adalah variabel yang didefinisikan dalam fungsi untuk menerima nilai dari luar.
- **Argumen** adalah nilai yang dikirim saat memanggil fungsi.

Contoh:

```
def luas_persegi(sisi): # sisi = parameter
    return sisi * sisi

print(luas_persegi(4)) # 4 = argumen
```

Output:

16

Return Value

- return digunakan untuk **mengembalikan nilai** dari fungsi.
- Setelah return, fungsi akan berhenti dieksekusi.

Contoh:

```
def tambah(a, b):
    return a + b

hasil = tambah(10, 5)
print("Hasil:", hasil)
```

Output:

Hasil: 15

Parameter dengan Nilai Default

Jika argumen tidak diberikan, maka parameter akan menggunakan nilai default.

```
def sapa(nama="User"):
    return f'Halo, {nama}!'

print(sapa())      # pakai default
print(sapa("Budi")) # pakai argumen
```

Output:

Halo, User!

Halo, Budi!

Parameter Lebih dari Satu

```
def hitung_luas(panjang, lebar):
    return panjang * lebar

print("Luas:", hitung_luas(5, 3))
```

Output:

Luas: 15

Mengembalikan Banyak Nilai

Python bisa return lebih dari satu nilai sekaligus (sebagai tuple).

```
def operasi_matematika(a, b):
    tambah = a + b
    kurang = a - b
    kali = a * b
    bagi = a / b
    return tambah, kurang, kali, bagi

hasil = operasi_matematika(10, 2)
print(hasil)      # (12, 8, 20, 5.0)
```



```
# Bisa juga di-unpack
t, k, kl, b = operasi_matematika(10, 2)
print("Tambah:", t)
print("Kurang:", k)
```

Ringkasan

- Parameter = variabel dalam fungsi untuk menerima input
- Return = nilai keluaran dari fungsi
- Bisa pakai **default parameter**
- Bisa return **lebih dari satu nilai**

Fungsi Built-in di Python

Apa itu Fungsi Built-in?

- **Fungsi built-in** adalah fungsi yang sudah tersedia di Python, bisa langsung digunakan tanpa perlu impor library.
- Membantu mempercepat pemrograman karena tidak perlu membuat fungsi dari nol.

Contoh Fungsi Built-in yang Sering Dipakai

1. **len()** → menghitung panjang data (jumlah elemen)

```
nama = "Python"
angka = [10, 20, 30, 40]

print(len(nama)) # 6 (jumlah karakter)
print(len(angka)) # 4 (jumlah elemen dalam list)
```

2. **max()** → mencari nilai terbesar

```
angka = [3, 7, 1, 9, 5]
print(max(angka)) # 9
```

3. **min()** → mencari nilai terkecil

```
angka = [3, 7, 1, 9, 5]
print(min(angka)) # 1
```

4. **sum()** → menjumlahkan seluruh elemen (numerik)

```
angka = [3, 7, 1, 9, 5]
```

```
print(sum(angka)) # 25
```

5. abs() → nilai absolut (positif)

```
print(abs(-10)) # 10  
print(abs(7)) # 7
```

6. round() → membulatkan angka

```
print(round(3.14159, 2)) # 3.14 (2 angka di belakang koma)  
print(round(7.5)) # 8
```

7. type() → mengetahui tipe data

```
print(type(10)) # <class 'int'>  
print(type(3.14)) # <class 'float'>  
print(type("Halo")) # <class 'str'>
```

8. sorted() → mengurutkan data

```
angka = [7, 3, 9, 1, 5]  
print(sorted(angka)) # [1, 3, 5, 7, 9]  
print(sorted(angka, reverse=True)) # [9, 7, 5, 3, 1]
```

9. range() → menghasilkan deret angka

```
for i in range(5):  
    print(i, end=" ") # 0 1 2 3 4
```

10. input() & print() → interaksi dengan pengguna

```
nama = input("Masukkan nama: ")  
print("Halo,", nama)
```

Ringkasan

- Fungsi built-in memudahkan pekerjaan pemrograman.
- Contoh: len(), max(), min(), sum(), abs(), round(), type(), sorted(), range(), input(), print().

Lambda Function (Fungsi Anonim)

Apa itu Lambda Function?

- **Lambda function** adalah fungsi kecil (anonim, tanpa nama) yang bisa ditulis dalam satu baris kode.

- Biasanya dipakai untuk operasi sederhana, terutama ketika kita **tidak perlu membuat fungsi dengan def.**

Bentuk Umum

lambda argumen: ekspresi

- lambda → kata kunci untuk membuat fungsi anonim
- argumen → parameter (bisa lebih dari satu)
- ekspresi → nilai yang dikembalikan

Contoh Sederhana

```
# Fungsi biasa
def tambah(a, b):
    return a + b

# Lambda function
tambah_lambda = lambda a, b: a + b

print(tambah(5, 3))    # 8
print(tambah_lambda(5, 3)) # 8
```

Lambda dengan Satu Parameter

```
kuadrat = lambda x: x ** 2
print(kuadrat(4)) # 16
```

Lambda Tanpa Parameter

```
say_hello = lambda: "Halo Python!"
print(say_hello()) # Halo Python!
```

Lambda dengan Beberapa Argumen

```
keliling_persegi = lambda s1, s2, s3, s4: s1 + s2 + s3 + s4
print(keliling_persegi(2, 3, 2, 3)) # 10
```

Lambda dalam sorted()

Biasanya lambda digunakan di fungsi bawaan seperti sorted(), map(), filter().

```
data = [("Budi", 25), ("Ani", 19), ("Cici", 22)]

# Urutkan berdasarkan usia (index 1)
urut = sorted(data, key=lambda x: x[1])
print(urut)
# [('Ani', 19), ('Cici', 22), ('Budi', 25)]
```

Lambda dengan map()

```
angka = [1, 2, 3, 4, 5]
kuadrat = list(map(lambda x: x**2, angka))
print(kuadrat) # [1, 4, 9, 16, 25]
```

Lambda dengan filter()

```
angka = [1, 2, 3, 4, 5, 6]
genap = list(filter(lambda x: x % 2 == 0, angka))
print(genap) # [2, 4, 6]
```

Ringkasan

- lambda digunakan untuk membuat fungsi singkat tanpa def.
- Cocok dipakai dengan fungsi built-in (sorted, map, filter, reduce).
- Biasanya dipakai untuk operasi sederhana.

PEMROGRAMAN BERORIENTASI OBJEK (OOP)

Apa itu OOP?

- **OOP (Object-Oriented Programming)** adalah paradigma pemrograman yang berfokus pada **objek**.
- Objek memiliki **atribut (data/properti)** dan **method (fungsi/perilaku)**.
- Konsep utama: **class, object, inheritance, encapsulation, polymorphism**.

Class

- **Class** adalah blueprint/cetakan untuk membuat objek.
- Berisi definisi atribut (variabel) dan method (fungsi).

Contoh:

```
class Mobil:
    # Atribut class
    roda = 4

    # Konstruktor (dipanggil saat membuat objek)
    def __init__(self, merk, warna):
        self.merk = merk
        self.warna = warna

    # Method
    def info(self):
        return f'Mobil {self.merk} berwarna {self.warna}'
```

Object

- **Object** adalah hasil instansiasi (pembuatan) dari class.
- Satu class bisa dipakai untuk membuat banyak object dengan data berbeda.

Contoh:

```
# Membuat objek dari class Mobil
mobil1 = Mobil("Toyota", "Hitam")
```

```
mobil2 = Mobil("Honda", "Merah")
```

```
# Mengakses method
```

```
print(mobil1.info())
```

```
print(mobil2.info())
```

Output:

Mobil Toyota berwarna Hitam

Mobil Honda berwarna Merah

Atribut

- Ada **atribut instance** → milik masing-masing objek
- Ada **atribut class** → dimiliki bersama oleh semua objek

```
print(mobil1.roda) # 4
```

```
print(mobil2.roda) # 4
```

Contoh Lain

```
class Mahasiswa:
```

```
    def __init__(self, nama, nim):
```

```
        self.nama = nama
```

```
        self.nim = nim
```

```
    def belajar(self, mata_kuliah):
```

```
        return f'{self.nama} sedang belajar {mata_kuliah}'
```

```
# Membuat objek
```

```
mhs1 = Mahasiswa("Budi", "12345")
```

```
mhs2 = Mahasiswa("Siti", "67890")
```

```
print(mhs1.belajar("Python"))
```

```
print(mhs2.belajar("Matematika"))
```

Output:

Budi sedang belajar Python

Siti sedang belajar Matematika

Ringkasan

- **Class** = blueprint cetakan objek
- **Object** = hasil nyata dari class
- **Atribut** = data yang dimiliki objek
- **Method** = fungsi/perilaku objek

Constructor (`__init__`) dalam Python

Apa itu Constructor?

- **Constructor** adalah method khusus dalam class Python yang otomatis dipanggil ketika kita membuat **objek baru**.
- Nama constructor di Python adalah `__init__`.
- Biasanya digunakan untuk **menginisialisasi atribut** pada objek.

Sintaks Dasar

```
class NamaClass:  
    def __init__(self, parameter1, parameter2, ...):  
        # Inisialisasi atribut  
        self.parameter1 = parameter1  
        self.parameter2 = parameter2
```

Keterangan:

- `__init__` → nama method khusus untuk constructor
- `self` → merepresentasikan objek itu sendiri (wajib ada sebagai argumen pertama)

Contoh 1: Constructor sederhana

```
class Mobil:  
    def __init__(self, merk, warna):  
        self.merk = merk    # atribut instance  
        self.warna = warna  
  
    def info(self):  
        return f'Mobil {self.merk} berwarna {self.warna}'  
  
# Membuat objek
```

```
mobil1 = Mobil("Toyota", "Hitam")
mobil2 = Mobil("Honda", "Merah")

print(mobil1.info())
print(mobil2.info())
```

Output:

Mobil Toyota berwarna Hitam

Mobil Honda berwarna Merah

Contoh 2: Constructor dengan nilai default

```
class Mahasiswa:
    def __init__(self, nama, nim="00000"):
        self.nama = nama
        self.nim = nim

m1 = Mahasiswa("Budi", "12345")
m2 = Mahasiswa("Siti") # nim akan default "00000"

print(m1.nama, m1.nim) # Budi 12345
print(m2.nama, m2.nim) # Siti 00000
```

Contoh 3: Constructor tanpa parameter tambahan

```
class Hewan:
    def __init__(self):
        print("Objek Hewan dibuat!")

h1 = Hewan() # otomatis memanggil constructor
```

Output:

Objek Hewan dibuat!

Kenapa Constructor Penting?

- Memudahkan pemberian nilai awal (inisialisasi) pada atribut objek.
- Mengurangi kode berulang.

- Membuat class lebih **terstruktur** dan **rapi**.

Method dan Property di Python OOP

1. Method

- **Method** adalah fungsi yang didefinisikan di dalam class.
- Selalu memiliki parameter pertama self untuk merepresentasikan objek.

Jenis Method:

1. **Instance Method** → method biasa, bekerja pada objek
2. **Class Method** → menggunakan @classmethod, bekerja pada class, bukan objek
3. **Static Method** → menggunakan @staticmethod, tidak terikat ke objek atau class

Contoh: Instance Method

```
class Mobil:
    def __init__(self, merk, warna):
        self.merk = merk
        self.warna = warna

    def info(self): # Instance Method
        return f'Mobil {self.merk} berwarna {self.warna}'

mobil1 = Mobil("Toyota", "Hitam")
print(mobil1.info())
```

Output:

Mobil Toyota berwarna Hitam

Contoh: Class Method

```
class Mahasiswa:
    jumlah_mahasiswa = 0 # atribut class

    def __init__(self, nama):
        self.nama = nama
        Mahasiswa.jumlah_mahasiswa += 1
```

```

@classmethod
def get_jumlah(cls): # Class Method
    return f"Total mahasiswa: {cls.jumlah_mahasiswa}"

m1 = Mahasiswa("Budi")
m2 = Mahasiswa("Siti")

print(Mahasiswa.get_jumlah())

```

Output:

Total mahasiswa: 2

Contoh: Static Method

```

class Matematika:
    @staticmethod
    def tambah(a, b):
        return a + b

print(Matematika.tambah(5, 3)) # Bisa dipanggil tanpa buat objek

```

Output:

8

2. Property

- **Property** adalah cara membuat atribut lebih aman dengan `@property`.
- Bisa digunakan untuk membuat atribut **read-only** atau menambahkan **validasi**.

Contoh: Property Read-Only

```

class Lingkaran:
    def __init__(self, jari2):
        self._jari2 = jari2

    @property
    def luas(self): # hanya bisa dibaca, tidak bisa diubah langsung
        return 3.14 * (self._jari2 ** 2)

```

```
ling = Lingkaran(7)
print(ling.luas) # 153.86
# ling.luas = 200 ❌ Error (read-only)
```

Contoh: Property dengan Setter

```
class Mahasiswa:
    def __init__(self, nama, umur):
        self._nama = nama
        self._umur = umur

    @property
    def umur(self):
        return self._umur

    @umur.setter
    def umur(self, value):
        if value < 0:
            print("Umur tidak boleh negatif!")
        else:
            self._umur = value

m1 = Mahasiswa("Budi", 20)
print(m1.umur) # 20
m1.umur = -5 # Tidak valid
m1.umur = 22 # Valid
print(m1.umur) # 22
```

Output:

20

Umur tidak boleh negatif!

22

Ringkasan

- **Method** → fungsi dalam class (instance method, class method, static method).
- **Property** → cara mengatur akses atribut agar lebih aman & terkontrol.



ENCAPSULATION & POLYMORPHISM (PENGANTAR)

1. Encapsulation

Encapsulation = **pembungkusan data** → menyembunyikan detail internal sebuah objek agar lebih aman.

Python tidak punya **access modifier** seperti public, private, protected di Java/C#, tapi ada **konvensi penamaan**:

- **Public** → bisa diakses dari mana saja (self.nama)
- **Protected** → hanya boleh diakses di dalam class & subclass (self._nama)
- **Private** → benar-benar tersembunyi, pakai __ di depannya (self.__nama)

Contoh Encapsulation:

```
class Mahasiswa:
    def __init__(self, nama, umur):
        self.nama = nama      # public
        self._jurusan = "TI"  # protected
        self.__ipk = 3.8      # private

    def tampilkan_info(self):
        return f>Nama: {self.nama}, Jurusan: {self._jurusan}, IPK: {self.__ipk}"

m1 = Mahasiswa("Budi", 20)
print(m1.nama)           # ✅ bisa diakses
print(m1._jurusan)       # ⚠️ bisa diakses tapi tidak dianjurkan
# print(m1.__ipk)        # ❌ Error (private)
print(m1.tampilkan_info())
```

Output:

Nama: Budi, Jurusan: TI, IPK: 3.8

`__ipk` hanya bisa diakses lewat method dalam class, bukan langsung dari luar.

2. Polymorphism

Polymorphism = **satu interface, banyak bentuk**. Artinya, method/objek yang sama bisa punya perilaku berbeda.

a. Polymorphism dengan Method Override (Inheritance)

```
class Hewan:
    def suara(self):
        return "Hewan bersuara"

class Kucing(Hewan):
    def suara(self):
        return "Meong"

class Anjing(Hewan):
    def suara(self):
        return "Guk guk"

hewan_list = [Kucing(), Anjing(), Hewan()]
for h in hewan_list:
    print(h.suara())
```

Output:

Meong

Guk guk

Hewan bersuara

b. Polymorphism dengan Fungsi Biasa

Python juga mendukung polymorphism lewat fungsi yang menerima **berbagai tipe objek**.

```
def cetak_info(obj):
    print(obj.info())

class Buku:
    def info(self):
        return "Ini buku"
```

```
class Majalah:
    def info(self):
        return "Ini majalah"

cetak_info(Buku()) # Ini buku
cetak_info(Majalah()) # Ini majalah
```

Ringkasan

- **Encapsulation** → menyembunyikan data dengan public, `_protected`, `__private`.
- **Polymorphism** → satu method bisa memiliki banyak bentuk (override, dynamic typing).

ERROR & EXCEPTION HANDLING

1. Jenis Error Umum di Python

Error di Python terbagi menjadi 2 jenis utama:

a. Syntax Error

Terjadi saat ada kesalahan penulisan kode.

```
print("Hello World" # lupa kurung tutup
```

Output:

SyntaxError: unexpected EOF while parsing

b. Runtime Error (Exception)

Terjadi saat program berjalan. Beberapa contoh umum:

1. **TypeError** → operasi dengan tipe data yang salah.

```
print("10" + 5)
```

Output:

TypeError: can only concatenate str (not "int") to str

2. **ValueError** → nilai tidak sesuai dengan tipe data.

```
int("abc")
```

Output:

ValueError: invalid literal for int() with base 10: 'abc'

3. **NameError** → variabel belum didefinisikan.

```
print(x)
```

Output:

NameError: name 'x' is not defined

4. **ZeroDivisionError** → pembagian dengan nol.

```
print(10 / 0)
```

Output:

ZeroDivisionError: division by zero

5. **IndexError** → mengakses index list yang tidak ada.

```
data = [1, 2, 3]
```



```
print(data[5])
```

Output:

IndexError: list index out of range

6. **KeyError** → mengakses key dictionary yang tidak ada.

```
d = {"nama": "Budi"}  
print(d["umur"])
```

Output:

KeyError: 'umur'

Error Handling di Python

1. try-except

Digunakan untuk menangani error agar program **tidak berhenti tiba-tiba**.

Contoh:

```
try:  
    x = int("abc") # ini akan error  
except ValueError:  
    print("Terjadi ValueError: input harus berupa angka")
```

Output:

Terjadi ValueError: input harus berupa angka

Multiple Except

Bisa menangani **beberapa jenis error**:

```
try:  
    angka = 10 / 0  
except ZeroDivisionError:  
    print("Tidak bisa membagi dengan nol")  
except ValueError:  
    print("Input salah")
```

Catch Semua Error

```
try:  
    print(10 / 0)  
except Exception as e:
```

```
print("Terjadi error:", e)
```

Output:

Terjadi error: division by zero

2. finally

finally akan **selalu dijalankan**, baik ada error atau tidak. Biasanya dipakai untuk menutup file/database.

```
try:
    f = open("data.txt", "r")
    print(f.read())
except FileNotFoundError:
    print("File tidak ditemukan")
finally:
    print("Blok finally selalu dieksekusi")
```

Output jika file tidak ada:

File tidak ditemukan

Blok finally selalu dieksekusi

3. raise

raise digunakan untuk **memunculkan error secara manual**.

Contoh:

```
def bagi(a, b):
    if b == 0:
        raise ZeroDivisionError("Pembagi tidak boleh nol")
    return a / b

print(bagi(10, 2)) #  berhasil
print(bagi(5, 0)) #  raise error
```

Output:

5.0

Traceback (most recent call last):

...

ZeroDivisionError: Pembagi tidak boleh nol

Ringkasan

- try-except → menangkap error.
- finally → selalu dijalankan.
- raise → memunculkan error manual.



MODUL & PACKAGE

1. Import Modul Bawaan

Python sudah menyediakan banyak **modul standar** yang bisa langsung digunakan tanpa install tambahan.

a. Modul math

```
import math

print(math.sqrt(16))    # akar kuadrat -> 4.0
print(math.pi)         # konstanta pi -> 3.14159...
print(math.factorial(5)) # faktorial -> 120
```

b. Modul datetime

```
import datetime

sekarang = datetime.datetime.now()
print("Sekarang:", sekarang)

hari_ini = datetime.date.today()
print("Tanggal hari ini:", hari_ini)
```

c. Modul random

```
import random

print(random.randint(1, 10)) # bilangan acak 1-10
print(random.choice(["apel", "mangga", "pisang"])) # pilih acak dari list
```

2. Membuat Modul Sendiri

Modul adalah file .py yang berisi fungsi, class, atau variabel, agar bisa digunakan ulang.

Contoh: Buat file mymath.py

```
# mymath.py
def tambah(a, b):
    return a + b

def kali(a, b):
    return a * b
```

Gunakan di file lain:

```
import mymath

print(mymath.tambah(3, 5)) # 8
print(mymath.kali(4, 6)) # 24
```

3. Package (Folder Modul)

Package = kumpulan modul dalam **satu folder**, biasanya ada file `__init__.py` di dalamnya.

Struktur

```
myproject/
|
|— mypackage/
|   |— __init__.py
|   |— modul1.py
|   |— modul2.py
|— main.py
```

Isi modul1.py

```
def sapa():
    print("Halo dari modul1")
```

Gunakan di **main.py**

```
from mypackage import modul1

modul1.sapa()
```

4. Menggunakan pip & Library Eksternal

pip adalah package manager Python untuk menginstal library eksternal.

Contoh install:

```
pip install requests
```

Contoh penggunaan:

```
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
print(response.json())
```

Ringkasan

- Modul bawaan: math, datetime, random, dll.
- Bisa membuat **modul sendiri** untuk digunakan ulang.
- Package = kumpulan modul dalam folder.
- pip → instalasi library eksternal seperti requests, numpy, pandas.

FILE HANDLING

Python menyediakan fungsi bawaan `open()` untuk bekerja dengan file (membaca & menulis).

1. Membuka File

Sintaks umum:

```
file = open("namafile.txt", "mode")
```

Setelah selesai, **jangan lupa menutup file** dengan `file.close()`. Atau lebih aman menggunakan `with open(...) as ...` (otomatis menutup file).

2. Membaca File Teks

a. `read()` → membaca seluruh isi file

```
with open("data.txt", "r") as f:
    isi = f.read()
    print(isi)
```

b. `readline()` → membaca satu baris

```
with open("data.txt", "r") as f:
    baris1 = f.readline()
    print(baris1)
```

c. `readlines()` → membaca semua baris dalam bentuk list

```
with open("data.txt", "r") as f:
    semua_baris = f.readlines()
    print(semua_baris) # ['baris1\n', 'baris2\n', ...]
```

3. Menulis File

a. `write()` → menulis string tunggal

```
with open("output.txt", "w") as f:
    f.write("Halo, ini baris pertama.\n")
    f.write("Ini baris kedua.\n")
```

b. `writelines()` → menulis list string sekaligus

```
with open("output.txt", "w") as f:
```

```
f.writelines(["Baris 1\n", "Baris 2\n", "Baris 3\n"])
```

4. Mode File

Mode	Keterangan
"r"	read (membaca, default). File harus ada .
"w"	write (menulis, membuat file baru / menimpa file lama).
"a"	append (menambah data di akhir file).
"rb"	read binary (membaca file biner, misalnya gambar).
"wb"	write binary (menulis file biner).

Contoh append:

```
with open("output.txt", "a") as f:  
    f.write("Tambahan baris baru.\n")
```

5. Contoh Membaca & Menulis Binary File

```
# Membaca gambar  
with open("gambar.jpg", "rb") as f:  
    data = f.read()  
  
# Menyalin gambar  
with open("copy.jpg", "wb") as f:  
    f.write(data)
```

Ringkasan:

- open() digunakan dengan mode (r, w, a, rb, wb).
- Membaca: read(), readline(), readlines().
- Menulis: write(), writelines().
- Gunakan with open(...) agar file otomatis tertutup.

TOPIK LANJUTAN PYTHON**1. List Comprehension**

Cara singkat untuk membuat list baru dengan ekspresi.

Contoh biasa:

```
angka = []  
for i in range(1, 6):  
    angka.append(i**2)  
print(angka) # [1, 4, 9, 16, 25]
```

Dengan **list comprehension**:

```
angka = [i**2 for i in range(1, 6)]  
print(angka) # [1, 4, 9, 16, 25]
```

Dengan kondisi:

```
genap = [i for i in range(10) if i % 2 == 0]  
print(genap) # [0, 2, 4, 6, 8]
```

2. Generator & Iterator

- **Iterator**: objek yang dapat diiterasi (for, next()).
- **Generator**: fungsi yang menghasilkan data **satu per satu** menggunakan yield.

Contoh iterator:

```
angka = iter([1, 2, 3])  
print(next(angka)) # 1  
print(next(angka)) # 2
```

Contoh generator:

```
def hitung(n):  
    for i in range(1, n+1):  
        yield i  
  
for x in hitung(5):
```

```
print(x)
# 1 2 3 4 5
```

Kelebihan generator: **hemat memori** karena data tidak disimpan sekaligus.

3. Decorators

Decorator adalah fungsi yang **membungkus fungsi lain** agar punya fitur tambahan tanpa mengubah kodenya.

Contoh sederhana:

```
def dekorator(fungsi):
    def wrapper():
        print("Sebelum fungsi dipanggil")
        fungsi()
        print("Setelah fungsi dipanggil")
    return wrapper

@dekorator
def halo():
    print("Halo, Python!")

halo()
```

Output:

Sebelum fungsi dipanggil

Halo, Python!

Setelah fungsi dipanggil

4. Virtual Environment

Digunakan untuk membuat **lingkungan Python terisolasi** agar library tiap proyek tidak bercampur.

Membuat virtual environment:

```
python -m venv env
```

Mengaktifkan:

- Windows:

```
env\Scripts\activate
```

- Linux/Mac:

```
source env/bin/activate
```

Menginstal library di dalam venv:

```
pip install requests
```

Menyimpan daftar library:

```
pip freeze > requirements.txt
```

Menginstal dari requirements.txt:

```
pip install -r requirements.txt
```

Ringkasan:

- **List comprehension** → cara singkat membuat list.
- **Generator** → menghasilkan data satu per satu (hemat memori).
- **Decorator** → menambahkan fitur ke fungsi lain.
- **Virtual Environment** → isolasi library antar proyek.

1. Membuat Program Kalkulator Sederhana

Menggunakan input, operator, dan percabangan.

```
def kalkulator():  
    print("=== Kalkulator Sederhana ===")  
    print("Operasi: +, -, *, /")  
  
    a = float(input("Masukkan angka pertama: "))  
    b = float(input("Masukkan angka kedua: "))  
    op = input("Pilih operator (+ - * /): ")  
  
    if op == "+":  
        hasil = a + b  
    elif op == "-":  
        hasil = a - b  
    elif op == "*":  
        hasil = a * b  
    elif op == "/":  
        hasil = a / b  
    else:  
        hasil = "Operator tidak valid!"  
  
    print("Hasil:", hasil)  
  
kalkulator()
```

2. Mengolah Data (CSV & JSON)

Membaca file CSV:

```
import csv
```

```
with open("data.csv", newline="") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Membaca & menulis JSON:

```
import json

# Menulis data ke JSON
data = {"nama": "Budi", "umur": 20, "jurusan": "Informatika"}
with open("data.json", "w") as f:
    json.dump(data, f)

# Membaca data dari JSON
with open("data.json", "r") as f:
    data_baca = json.load(f)
    print(data_baca)
```

3. Membuat API Request (dengan requests)

Install modul requests:

```
pip install requests
```

Contoh ambil data dari API publik:

```
import requests

url = "https://jsonplaceholder.typicode.com/todos/1"
res = requests.get(url)

if res.status_code == 200:
    data = res.json()
    print("Judul:", data["title"])
else:
    print("Gagal mengambil data API")
```

4. Mini Project: Aplikasi To-Do List

```
todo_list = []

def tampilkan():
    print("\n=== To-Do List ===")
    if not todo_list:
        print("Belum ada tugas.")
    else:
        for i, tugas in enumerate(todo_list, start=1):
            print(f'{i}. {tugas}')

def tambah():
    tugas = input("Masukkan tugas baru: ")
    todo_list.append(tugas)
    print("Tugas ditambahkan!")

def hapus():
    tampilkan()
    nomor = int(input("Pilih nomor tugas yang ingin dihapus: "))
    if 0 < nomor <= len(todo_list):
        todo_list.pop(nomor-1)
        print("Tugas berhasil dihapus!")
    else:
        print("Nomor tidak valid!")

while True:
    print("\nMenu: [1] Lihat [2] Tambah [3] Hapus [4] Keluar")
    pilihan = input("Pilih menu: ")

    if pilihan == "1":
        tampilkan()
    elif pilihan == "2":
        tambah()
```

```
elif pilihan == "3":  
    hapus()  
elif pilihan == "4":  
    print("Sampai jumpa!")  
    break  
else:  
    print("Pilihan tidak valid!")
```



Python banyak dipakai dalam bidang **Data Science**, **Web Development**, dan **Desktop GUI**. Berikut pengenalan singkatnya:

1. Data Science dengan Pandas, NumPy, Matplotlib

a. NumPy → operasi matematika & array

```
import numpy as np

# Membuat array
arr = np.array([1, 2, 3, 4, 5])
print("Array:", arr)

# Operasi
print("Rata-rata:", np.mean(arr))
print("Akar kuadrat:", np.sqrt(arr))
```

b. Pandas → analisis data (CSV, Excel, dsb.)

```
import pandas as pd

# Membaca data dari CSV
df = pd.read_csv("data.csv")
print(df.head()) # tampilkan 5 baris pertama

# Analisis sederhana
print("Rata-rata umur:", df["umur"].mean())
```

c. Matplotlib → visualisasi data

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
```



```
plt.plot(x, y, marker="o")
plt.title("Grafik Contoh")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

2. Web Development dengan Flask / FastAPI

a. Flask → framework web sederhana

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello, Flask!"

if __name__ == "__main__":
    app.run(debug=True)
```

Jalankan → buka <http://127.0.0.1:5000/>

b. FastAPI → API modern & cepat

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}
```

Jalankan dengan:

uvicorn main:app --reload

Lalu buka <http://127.0.0.1:8000/>

3. GUI (Graphical User Interface) dengan PyQt / Tkinter

a. Tkinter (bawaan Python, ringan)

```
import tkinter as tk

root = tk.Tk()
root.title("Aplikasi GUI Sederhana")

label = tk.Label(root, text="Hello, Tkinter!", font=("Arial", 14))
label.pack(pady=20)

button = tk.Button(root, text="Tutup", command=root.quit)
button.pack()

root.mainloop()
```

b. PyQt (lebih modern & profesional)

```
from PyQt5.QtWidgets import QApplication, QLabel

app = QApplication([])
label = QLabel("Hello, PyQt5!")
label.show()
app.exec_()
```

Dengan pengantar ini:

- Kalau minat Data Science → kuasai Pandas, NumPy, Matplotlib.
- Kalau minat Web Development → pelajari Flask / FastAPI.
- Kalau minat Aplikasi Desktop → bisa coba Tkinter / PyQt.